

BANA 622 - Programming for Business Analytics Financial API Project INTC

Julie Loyez

Executive summary	4
Business problem and user scenario	4
API source and endpoint explanation	4
Data dictionary for selected field	5
Business rules and algorithm explanation	5
Business Rules	5
Price Movement Rule	5
Volatility Rule	6
Signal Rule	6
Data Quality Rule	7
Algorithm explanation	7
Pseudocode	8
Flowchart	9
Python implementation overview	9
Data Retrieval	9
Response Validation and Parsing	10
Data Extraction and Metric Calculation	10
Volatility Classification	10
Signal Generation	10
Database Storage	10
Reporting	10
Database design and table schema	10
SQL queries and results	11
Query 1 - Count of Buy, Sell, and Hold signals	11
Query 2 - Average percent change by signal type	12

Query 3 - Highest volatility day	12
Query 4 - All High volatility days	12
Query 5 - Average closing price over the full period	13
Query 6 - Count of days by volatility category	13
Query 7 - Top 5 best performing days	13
Query 8 - Top 5 worst performing days	14
Final insights and recommendations	14
Overall Market Behavior	14
Volatility Fundings	14
Signal Findings	15
Recommendation for Business Stakeholders	15
Limitations and ethical note	15
Technical Limitations	15
Signal Logic Limitations	15
Ethical Note	16
Reflection	16
What Was Difficult	16
What Was Learned	16
What Could Be Improved	16
Appendix	17
Appendix 1 - Flowchart	17
Appendix 2 - Screenshot of the code running	18

Executive summary

This project presents a Python-based stock monitoring system designed to automatically retrieve, process, and analyze daily price for Intel Corporation (INTC).

INTC was selected as the target stock due to its exceptional volatility and ongoing strategic transition. Over the 100-day analysis period ending to May 8, 2026, the stock more than tripled in value, rising from a low close of \$39.37 to \$124.92, while experiencing High volatility on 52 out of 100 trading days.

The system retrieves daily price data from the Alpha Vantage API, calculates key metrics including daily percent change and daily range, classifies each day's volatility as Low, Moderate, or High, and generates a Buy, Sell, or Hold signal based on a set of clearly defined business rules. All processed records are stored in a SQLite database and summarized in a plain language report designed for nontechnical stakeholders.

Over the analysis period, the system generated 16 Buy signals, 35 Sell signals, and 49 Hold signals. The most recent signal is Hold, reflecting positive recent price momentum offset by persistently high daily volatility. The system recommends monitoring INTC closely and waiting for a period of lower volatility before taking a Buy position.

Business problem and user scenario

Intel Corporation (INTC) is one of the world's largest semiconductor manufacturers and a central player in the global technology supply chain. Once the undisputed leader of its industry, it has faced a "perfect storm" of manufacturing setbacks, competitive losses, and missed opportunities in the AI boom. INTC fell behind in developing smaller, faster chips, giving competitors like TSMC, Samsung and AMD the chance to pull ahead in both performance and market share. INTC also missed the generative AI wave, declining an early investment in OpenAI and failing to develop a chip that could compete with Nvidia's AI hardware. On the financial side, the company lost \$7 billion in its foundry division and its stock dropped over 50%, leading to its removal from the Dow Jones Industrial Average in the end of 2024. Because of this ongoing uncertainty and high volatility, Intel's stock is a strong candidate for a rules-based monitoring system.

Our small analytics team has been asked to build a Python-based monitoring system for Intel (INTC) stock. The system will automatically retrieve daily price data, calculate key metrics, classify stock behavior, and generate buy/sell/hold signals. Results will be stored in a database and summarized in plain language so that nontechnical stakeholders - such as management team or executive committee - can quickly understand what the stock is doing and what the system recommends, without needing to interpret data directly.

API source and endpoint explanation

To retrieve INTC's stock data, we used the Alpha Vantage API with the *TIME_SERIES_DAILY* function. This function returns daily stock price data for a selected stock, including open, high, low, close, and volume for each trading day, going back up to 20 years. The base URL for all requests is <https://www.alphavantage.co/query>.

Each API request requires three parameters:

- *function*: tells the API what type of data to return;
- *symbol*: sets to INTC for Intel;
- *api key*: authenticates the request using our team's personal key.

The API returns data in JSON format. The daily price records are stored inside a key called “*Time Series (Daily)*”, where each entry is organized by date. For each trading day, we extract five fields: the opening price, the highest price of the day, the lowest price of the day, the closing price, and the total volume of shares traded. These five fields form the foundation for all the calculations and signals we build in the following steps.

Data dictionary for selected field

The following table defines each field extracted from the Alpha Vantage API and used throughout the system:

Field	Data Type	Description
date	String	The trading day the record belongs to
open	Float	Stock price at market open
high	Float	Highest price reached during the day
low	Float	Lowest price reached during the day
close	Float	Stock price at market close
volume	Integer	Number of shares traded that day
percent_change	Float	Daily price change as a percentage of open price
daily_range	Float	Difference between high and low
volatility_label	String	Volatility classification for the day
signal	String	Buy, Sell, or Hold recommendation

The first six fields come directly from the API. The last four are calculated by our system and stored in the database alongside the raw price data.

Business rules and algorithm explanation

Business Rules

Our system applies four types of rules to each trading day: a price movement rule, a volatility rule, a signal rule, and a data quality rule.

Price Movement Rule

The price movement rule classifies each trading day as Positive, Negative, or Neutral based on the daily percent change, calculated as the difference between the closing price and the open price divided by the opening price.

Classification	Rule
Positive	Percentage change > 1%

Classification	Rule
Neutral	Percent change between -1% and 1%
Negative	Percent change < -1%

A threshold of 1% was chosen to filter out minor fluctuations that are not meaningful for decision-making. Days where the price barely moved are classified as Neutral rather than triggering a signal.

Volatility Rule

The volatility rule classifies each trading day as Low, Moderate, or High based on the daily range percentage, calculated as the difference between the high and low price divided by the opening price.

Classification	Rule
Low	Daily range percent < 2%
Moderate	Daily range percent between 2% and 5%
High	Daily range percent > 5%

These thresholds are justified by INTC's 30-day historical data. The average daily range over the analysis period was 4.17, with a standard deviation of 1.81, placing the moderate band comfortably around the mean. The overall annualized volatility for INTC over the 100-day analysis period was calculated using the standard deviation of daily returns multiplied by $\sqrt{252}$, which is the standard financial method for annualizing daily volatility.

Signal Rule

The signal rule combines the price movement classification and the daily volatility classification to generate a Buy, Sell, or Hold recommendation for each trading day.

Signal	Rule
Buy	Movement is Positive AND day volatility is not High
Sell	Movement is Negative
Hold	Movement is Neutral OR day volatility is High

The risk control layer is an important feature of this system: even on days where the price increased, a Buy signal is blocked if the daily volatility is High. This reflects the principle that large intraday swings indicate uncertainty and elevated risk, making it unwise to recommend buying even when the closing price is higher than the opening price.

Data Quality Rule

If the API response does not contain the expected “Time Series (Daily)” key (e.g., due to a rate limit, an invalid API key, or a network failure), the system stops immediately and prints a clear error message. No calculations, classifications, or database insertions are performed until valid data is confirmed. This is handled by the *validate_response()* function.

Algorithm explanation

1. Set the stock symbol to INTC and store the Alpha Vantage API key
2. Build the request URL using the base address, the function name, the symbol, and the API key as separate parameters
3. Send the GET request to the Alpha Vantage server
 1. If the request fails due to a connection error or timeout, print an error message and stop
4. Check that the server returned status code 200
 1. If not, print an error and stop
5. Parse the response as a JSON dictionary
 1. If parsing fails, print an error and stop
6. Check that the key “Time Series (Daily)” exists in the dictionary
 1. If it is missing, print an error and stop
7. Extract the daily time-series object from the validated JSON
8. Take the 100 most recent trading dates from the time series
9. For each date, read the open, high, low, close, and volume values and convert them from strings to numbers
10. Calculate the daily price change (close price - open price)
11. Calculate the daily percent change (price change / open price * 100)
12. Calculate the daily range (high price - low price)
13. After processing all 100 days, calculate the standard deviation of the daily percent changes
14. Calculate the annualized volatility percentage (standard deviation * square root of 252)
 1. If annualized volatility is below 25%, classify the overall period as Low volatility
 2. If annualized volatility is between 25% and 50%, classify the overall period as Moderate volatility
 3. If annualized volatility is above 50%, classify the overall period as High volatility
15. For each individual day, calculate the range percentage (daily range / open price * 100)
 1. If the range percentage is above 5%, label that day as High volatility
 2. If the range percentage is between 2% and 5%, label that day as Moderate volatility
 3. If the range percentage is below 2%, label that day as Low volatility
16. For each day, check the percent change to determine price movement
 1. If the price movement is above +1%, label as Positive
 2. If the price movement is below -1%, label as Negative
 3. If the price movement is between -1% and +1%, label as Neutral
17. Create Buy, Sell, Hold signal for each day
 1. If movement is Positive and the day’s volatility label is not High, assign a Buy signal
 2. If movement is Negative, assign a Sell signal
 3. In all other cases, including Positive movement on a High-volatility day and all Neutral days, assign a Hold signal
18. Connect to the SQLite database file and create the table if it does not already exist
19. For each processed record, insert the symbol, date, prices, volume, percent change, volatility label, and signal into the table
20. Use INSERT OR REPLACE with a primary key on symbol and date to prevent duplicate records
21. Run SQL queries:
 1. Count signals by type
 2. Calculate average percent change per signal

3. Identify the most volatile day
4. List all High-volatility days
5. Compute the average closing price
6. Count days per volatility category
7. Find the top and bottom five performing days
22. Print the final business report showing the price summary, volatility classification, signal breakdown, and the most recent day's recommendation
23. Close the database connection and end

Pseudocode

START

Set stock symbol to INTC

Set API key to personal Alpha Vantage key

Build API request URL using symbol, function, and API key

Send GET request to Alpha Vantage API

IF request fails due to connection error or timeout THEN

 Print error message and stop

IF response status code is not 200 THEN

 Print error message and stop

IF response does not contain Time Series data THEN

 Print rate limit or invalid key message and stop

Convert JSON response into Python dictionary

Extract the 100 most recent daily records

FOR each trading day in the records:

 Extract open, high, low, close, and volume as numeric values

 Calculate daily price change = close - open

 Calculate daily percent change = (price change / open) * 100

 Calculate daily range = high - low

 Calculate daily range percent = (daily range / open) * 100

 IF percent change > 1% THEN movement = Positive

 ELSE IF percent change < -1% THEN movement = Negative

 ELSE movement = Neutral

 IF daily range percent > 5% THEN day volatility = High

 ELSE IF daily range percent >= 2% THEN day volatility = Moderate

 ELSE day volatility = Low

 IF movement is Positive AND day volatility is not High THEN signal = Buy

 ELSE IF movement is Negative THEN signal = Sell

 ELSE signal = Hold

 Store record with all fields in list

Calculate standard deviation of daily percent changes

Calculate annualized volatility = standard deviation * $\sqrt{252}$
Classify overall volatility as Low, Moderate, or High

Connect to SQLite database
Create table if it does not exist
FOR each processed record:
 Insert symbol, date, open, high, low, close, volume,
 percent change, volatility label, and signal
 Use PRIMARY KEY to prevent duplicate entries
Commit and close database connection

Run SQL summary queries:
 Count Buy, Sell, and Hold signals
 Calculate average percent change by signal type
 Find highest volatility day
 List all High volatility days
 Calculate average closing price
 Count days by volatility category
 Find top 5 best performing days
 Find top 5 worst performing days

Print business summary report with:
 Analysis period and total days
 Price summary
 Volatility summary and interpretation
 Signal summary
 Most recent day recommendation

END

Flowchart

To see the flowchart, see Appendix 1.

The system follows a linear process that begins with building and sending the API request, validating the response, and extracting the daily records. For each trading day, the flowchart shows the sequential application of the price movement rule, the volatility rule, and the signal rule, before storing the results in the database and printing the final report. Each decision point includes a clear branch for error conditions, such as a failed API request or missing data.

Python implementation overview

The system is built as a single Python script organized into different functions, each with a clear purpose, input, and output. The script runs sequentially from top to bottom, with each function passing its results to the next.

Data Retrieval

The *build_api_url()* function constructs the API request URL using three parameters: the stock symbol, the function type, and the API key. The *make_basic_request()* function sends the GET request to Alpha Vantage and returns the response object. Error handling is built into this function to catch connection errors, timeouts, and unexpected failures.

Response Validation and Parsing

The *validate_response()* function checks that the response is valid before any processing begins. It verifies the status code, confirms the response is in JSON format, and checks that the expected “Time Series (Daily)” key is present. If any check fails, the program stops immediately. The *preview_json_structure()* function then converts the response into a Python dictionary, and *inspect_top_level_structure()* confirms the structure of the data.

Data Extraction and Metric Calculation

The *extract_daily_records()* function loops through the 100 most recent trading days and extracts the open, high, low, close, and volume fields for each day, converting all values from strings to numbers. The *calculate_metrics()* function then computes the daily price change, daily percent change, and daily range for each record, as well as 100-day averages for each metric.

Volatility Classification

The *calculate_annualized_volatility()* function computes the standard deviation of daily percent changes and multiplies it by $\sqrt{252}$ to produce an annualized volatility figure. The *classify_volatility()* function then assigns an overall volatility label of Low, Moderate, or High based on defined thresholds.

Signal Generation

The *generate_signal()* function loops through all records and applies the price movement and daily volatility rules to generate a Buy, Sell, or Hold signal for each trading day. A risk control layer blocks Buy signals on days where daily volatility is classified as High, even if the price increased.

Database Storage

The *create_database()* function connects to a SQLite database and creates the *intc_daily_prices* table if it does not already exist. The *insert_records()* function inserts all 100 processed records into the table, using a composite key on symbol and date to prevent duplicate entries.

Reporting

The *print_summary_report()* function prints a plain language summary of the analysis period, price behavior, volatility classification, signal distribution, and a recommendation for the most recent trading day. The report is designed to be readable by a nontechnical audience without requiring any knowledge of finance or programming.

Database design and table schema

The system uses a SQLite database to store all processed daily records for INTC stock. The database contains one table called *intc_daily_prices*. Each row represents one trading day and includes both the raw price data retrieved from the API and the calculated fields produced by the analytics system.

Column	Data Type	Description
symbol	Text	Stock ticker symbol (INTC)
trade_date	Text	Trader day in YYYY-MM-DD format
open_price	Real	Opening price for the day
high_price	Real	Highest price reached during the day
low_price	Real	Lowest price reached during the day
close_price	Real	Closing price for the day
volume	Integer	Number of shares traded
percent_change	Real	Daily percent change from open to close
volatility_label	Text	Low, Moderate, or High
signal	Text	Buy, Sell, or Hold

The table uses a composite primary key on *symbol* and *trade_date*, which means the combination of these two fields must be unique. This prevents duplicate records from being inserted if the program is run multiple times. The *INSERT OR REPLACE* statement is used during insertion, so if a record for the same symbol and date already exists, it is updated rather than duplicated.

SQL queries and results

The following queries were run directly in Python against the *intc_daily_prices* SQLite database after all 100 records were inserted.

Query 1 - Count of Buy, Sell, and Hold signals

```
SELECT signal, COUNT(*) AS count
FROM intc_daily_prices
GROUP BY signal
ORDER BY count DESC;
```

Output:

Signal	Count
Hold	49 days
Sell	35 days
Buy	16 days

The majority of days resulted in a Hold signal, reflecting the high level of daily volatility INTC experienced over the analysis period. Only 16 days met the conditions for a Buy signal, while 35 days triggered a Sell signal, consistent with INTC's difficult market performance over the past several months.

Query 2 - Average percent change by signal type

```
SELECT signal, ROUND(AVG(percent_change), 4) AS avg_percent_change
FROM intc_daily_prices
GROUP BY signal
ORDER BY avg_percent_change DESC;
```

Output:

Signal	Average Percent Change
Hold	3.28%
Buy	2.09%
Sell	-3.27%

Hold days show a higher average percent change than Buy days because many large positive price swings were blocked by the High volatility rule and classified as Hold instead of Buy. This confirms that the risk control layer is working as intended - filtering out days where the price moved sharply but under uncertain conditions.

Query 3 - Highest volatility day

```
SELECT trade_date, close_price, percent_change, volatility_label
FROM intc_daily_prices
ORDER BY ABS(percent_change) DESC
LIMIT 1;
```

Output:

Date	Close	Change	Volatility
2026-05-08	\$124.92	11.73%	High

The most volatile day in the analysis period was May 8, 2026, when INTC's stock jumped 11.73% in a single day. Despite the large price increase, the system correctly assigned a Hold signal due to the extremely high daily volatility.

Query 4 - All High volatility days

```
SELECT trade_date, close_price, percent_change, volatility_label
FROM intc_daily_prices
WHERE volatility_label = 'High'
ORDER BY trade_date DESC;
```

Output:

52 days

52 out of 100 trading days were classified as High volatility, confirming that INTC's stock has been in an unusually turbulent period. This is consistent with the overall annualized volatility classification and reflects the ongoing uncertainty surrounding the company's strategic transition.

Query 5 - Average closing price over the full period

```
SELECT ROUND(AVG(close_price), 2) AS avg_close_price
FROM intc_daily_prices;
```

Output:

Average Close Price
\$54.84

The average closing price over the 100-day analysis period was \$54.84. This reflects INTC's significant stock recovery from its late 2024 lows, though the wide range between the lowest close of \$39.37 and the most recent close of \$124.92 shows how dramatically the stock has moved over this period.

Query 6 - Count of days by volatility category

```
SELECT volatility_label, COUNT(*) AS count
FROM intc_daily_prices
GROUP BY volatility_label
ORDER BY count DESC;
```

Output:

Volatility	Count
High	52 days
Moderate	46 days
Low	2 days

The near-absence of Low volatility days - only 2 out of 100 - confirms that INTC has been an exceptionally active stock over this period. The roughly equal split between High and Moderate days reflects a stock in transition, with frequent large price swings mixed with more normal trading days.

Query 7 - Top 5 best performing days

```
SELECT trade_date, open_price, close_price, percent_change, signal
FROM intc_daily_prices
ORDER BY percent_change DESC
LIMIT 5;
```

Output:

Date	Open	Close	Change	Signal
2026-05-08	\$111.81	\$124.92	11.73%	Hold
2026-04-29	\$86.13	\$94.75	10.00%	Hold
2026-04-02	\$46.06	\$50.38	9.38%	Hold

Date	Open	Close	Change	Signal
2026-01-09	\$41.82	\$45.55	8.91%	Hold
2026-01-21	\$50.32	\$54.25	7.81%	Hold

All five best performing days received a Hold signal rather than a Buy. This is because all five days also had High day volatility, triggering the risk control rule. While these were strong price days, the large intraday swings indicated elevated uncertainty, and the system correctly avoided recommending a Buy under those conditions.

Query 8 - Top 5 worst performing days

```
SELECT trade_date, open_price, close_price, percent_change, signal
FROM intc_daily_prices
ORDER BY percent_change ASC
LIMIT 5;
```

Output:

Date	Open	Close	Change	Signal
2026-03-20	\$46.95	\$43.87	-6.56%	Sell
2026-02-10	\$50.41	\$47.13	-6.51%	Sell
2026-03-30	\$43.72	\$41.19	-5.79%	Sell
2026-01-05	\$41.59	\$39.37	-5.34%	Sell
2026-01-30	\$48.97	\$46.47	-5.11%	Sell

All five worst performing days correctly received a Sell signal, confirming that the system reliably identifies significant negative price movement. The worst day was March 20, 2026, with a drop of 6.56%.

Final insights and recommendations

Overall Market Behavior

Over the 100-day analysis period ending May 8, 2026, INTC's stock showed exceptional volatility and a strong upward trend. The stock rose from a low close of \$39.37 to a recent close of \$124.92, a gain of over 200%. The average closing price of \$54.84 reflects the fact that much of this growth was concentrated in the most recent weeks of the analysis period, with the stock trading at significantly lower levels for most of early 2026.

Volatility Fundings

The system classified INTC's overall volatility as High, with an annualized volatility well above the 50% threshold. This is confirmed by the query results, which showed that 52 out of 100 trading days were classified as High volatility and only 2 days as Low volatility. This level of volatility is unusual even for a technology stock and reflects the ongoing uncertainty surrounding INTC's strategic direction, competitive position, and financial performance.

Signal Findings

The signal distribution - 49 Hold, 35 Sell, and 16 Buy - tells an important story about INTC's recent performance. The high number of Sell signals in the early part of the analysis period reflects the stock's difficult start to 2026, while the more recent Buy and Hold signals reflect a sharp recovery. The risk control layer proved particularly effective: all five of the best performing days were classified as Hold rather than Buy because of their extremely high daily volatility, demonstrating that the system prioritizes risk management over chasing price gains.

Recommendation for Business Stakeholders

Based on the most recent trading day in the analysis period, the system generated a Hold signal. While INTC's stock has shown remarkable recovery over the past several months, the persistently high volatility suggests that conditions remain uncertain. The system recommends monitoring the stock closely and waiting for a combination of positive price movement and moderate volatility before considering a Buy position.

As a summary, we can say that INTC's stock has been moving fast and in large swings. The analytics system is designed to flag the right moment to act, and right now, it is recommending patience.

Limitations and ethical note

Technical Limitations

The system has several technical limitations that should be considered when interpreting its results.

The free Alpha Vantage API key limits data retrieval to 25 requests per day and returns a maximum of 100 daily records per request without a premium subscription. This means the analysis is based on a relatively short window of 100 trading days, which may not capture longer-term trends or seasonal patterns in INTC's stock behavior.

The system retrieves data once per session and does not update in real time. In a production environment, a more robust system would pull data continuously throughout the trading day and recalculate signals as new price information becomes available.

The volatility and signal thresholds were designed especially for INTC based on its 100-day historical data. These thresholds may not be appropriate for other stocks with different volatility profiles, and would need to be recalibrated before applying the system to a different company.

Signal Logic Limitations

The buy/sell/hold signal is based solely on daily price movement and intraday volatility. It does not take into account any external factors that are known to significantly influence stock prices, such as earnings announcements, macroeconomic indicators, interest rate decisions, geopolitical events, or company-specific news. A more complex system would incorporate these factors into its decision logic.

The signal logic also looks at each day in isolation and does not consider multi-day trends. For example, a stock that has been declining for ten consecutive days but shows a small positive movement on day eleven would still receive a Buy signal under the current rules. A more sophisticated system would incorporate trend analysis over multiple days before generating a signal.

Ethical Note

The signals generated by this system are not real financial advice and should not be used to make actual investment decisions. Stock market investing carries significant financial risk, and no algorithmic system can reliably predict future price movements.

The system does not account for an individual's financial situation, risk tolerance, investment horizon, or personal goals. Anyone considering investing in INTC or any other stock should consult a licensed financial advisor before making any decisions.

Reflection

What Was Difficult

The most challenging part of this project was designing the API data pipeline and making sure each step passed the right data to the next function cleanly. From the beginning, variables like *data* and *records* were not always in scope when needed, which caused several errors that required debugging. Learning to structure the code so that each function has a clear input, output, and purpose, was a significant learning curve but ultimately made the code much easier to manage.

Setting the volatility thresholds was also more complex than expected. Rather than picking arbitrary numbers, we grounded the thresholds in INTC's own historical data using standard deviation and annualized volatility, which required understanding both the financial concepts and how to implement them in Python. Connecting the statistical logic to the business rules in a way that was both defensible and easy to explain to a nontechnical audience was a challenge.

Finally, working with the SQLite database and making sure records were inserted correctly without duplicates was challenging, giving extra attention to the table schema and the use of composite primary keys.

What Was Learned

This project brought together every major topic covered in this and previous courses - APIs, JSON parsing, Python functions, loops, conditionals, SQL databases, and business communication - into a single working system. Seeing how each piece connects to the others made the individual concepts feel much more meaningful than they did in isolation.

Working with a real API and real stock data also made the project feel practical and relevant. Understanding how financial data is structured, retrieved, and processed is directly applicable to a career in business analytics, particularly in industries like finance, technology, and entertainment where data-driven decision making is increasingly important.

The project also reinforced the importance of error handling. Building validation checks at every step — for the API request, the JSON response, and the database insertion — made the system much more robust and professional.

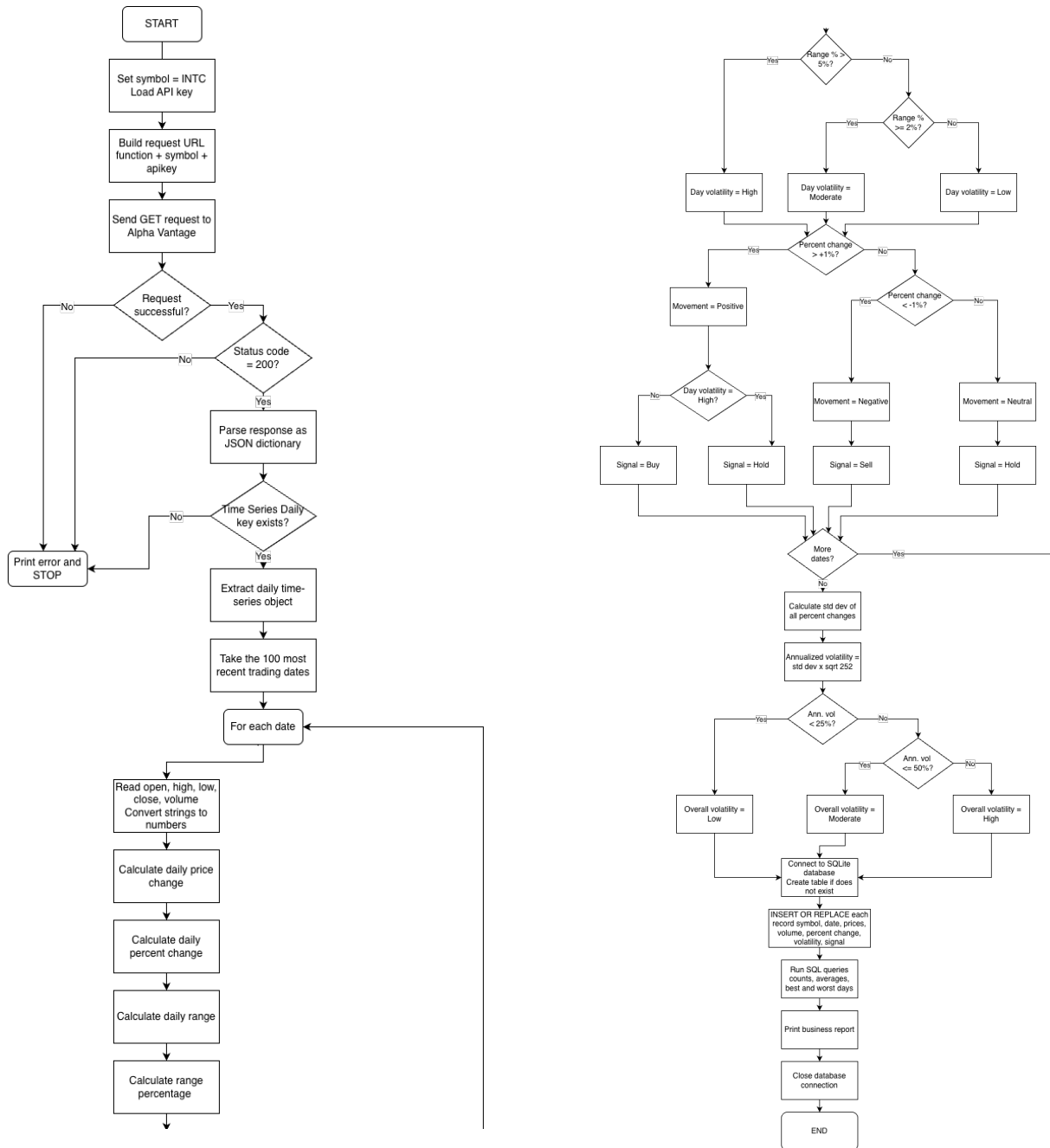
What Could Be Improved

With more time and resources, several improvements could be made to the system. A premium API key would allow access to more historical data, enabling longer-term trend analysis and more statistically reliable volatility calculations. The signal logic could be expanded to incorporate multi-day trend analysis, moving averages, or external news sentiment data to produce more nuanced recommendations.

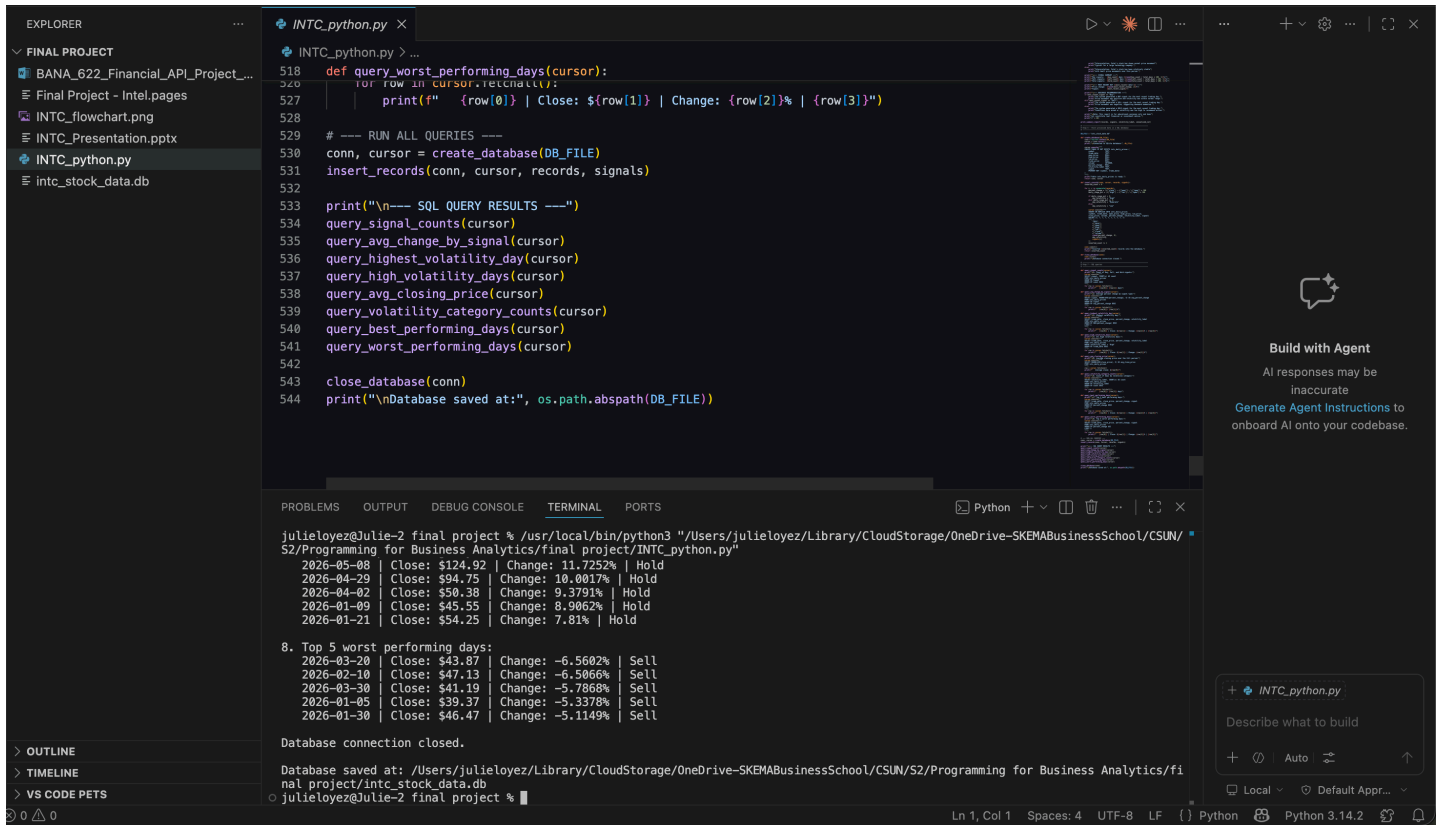
The system could also be made interactive, allowing a user to enter any stock symbol and receive a real-time analysis report, rather than being hardcoded for INTC. This would make it a genuinely reusable analytics tool rather than a single-stock demonstration.

Appendix

Appendix 1 - Flowchart



Appendix 2 - Screenshot of the code running



The screenshot shows a VS Code editor with a Python script named `INTC_python.py` open. The script is running, and the output is visible in the terminal. The script connects to a database, runs queries, and prints results. The terminal output shows the execution of the script and the resulting data.

```
def query_worst_performing_days(cursor):
    for row in cursor.fetchall():
        print(f" {row[0]} | Close: ${row[1]} | Change: {row[2]}% | {row[3]}")

# --- RUN ALL QUERIES ---
conn, cursor = create_database(DB_FILE)
insert_records(conn, cursor, records, signals)

print("\n--- SQL QUERY RESULTS ---")
query_signal_counts(cursor)
query_avg_change_by_signal(cursor)
query_highest_volatility_day(cursor)
query_high_volatility_days(cursor)
query_avg_closing_price(cursor)
query_volatility_category_counts(cursor)
query_best_performing_days(cursor)
query_worst_performing_days(cursor)

close_database(conn)
print("\nDatabase saved at:", os.path.abspath(DB_FILE))
```

Terminal Output:

```
juliellopez@Julie-2 final project % /usr/local/bin/python3 "/Users/juliellopez/Library/CloudStorage/OneDrive-SKEMA Business School/CSUN/S2/Programming for Business Analytics/final project/INTC_python.py"
2026-05-08 | Close: $124.92 | Change: 11.7252% | Hold
2026-04-29 | Close: $94.75 | Change: 10.0017% | Hold
2026-04-02 | Close: $50.38 | Change: 9.3791% | Hold
2026-01-09 | Close: $45.55 | Change: 8.9062% | Hold
2026-01-21 | Close: $54.25 | Change: 7.81% | Hold

8. Top 5 worst performing days:
2026-03-20 | Close: $43.87 | Change: -6.5602% | Sell
2026-02-10 | Close: $47.13 | Change: -6.5066% | Sell
2026-03-30 | Close: $41.19 | Change: -5.7868% | Sell
2026-01-05 | Close: $39.37 | Change: -5.3378% | Sell
2026-01-30 | Close: $46.47 | Change: -5.1149% | Sell

Database connection closed.
Database saved at: /Users/juliellopez/Library/CloudStorage/OneDrive-SKEMA Business School/CSUN/S2/Programming for Business Analytics/final project/intc_stock_data.db
juliellopez@Julie-2 final project %
```